

FLIT: A Lattice-Based Key Encapsulation Mechanism

Algorithm Specification And Supporting Documentation

Yanbin Pan, Yihui Wu, Qian Lu, Chi Cheng, Renzhang Liu, Qian Du

June 29, 2026

Contents

1	Algorithm Specification	3
1.1	Preliminaries and Notation	3
1.2	Homomorphic Encoding and Decoding.	7
1.3	Specification	9
1.3.1	Specification of FLIT.CPAPKE	10
1.3.2	Specification of FLIT.CCAKEM	10
1.4	Correctness	11
1.5	FLIT Parameter Sets	12
2	Design Rationale	13
3	Security Analysis	14
3.1	Theoretical Security	14
3.1.1	Hardness Assumption	14
3.1.2	Security Definition	15
3.1.3	Security Reduction	15
3.2	Concrete Security	17
3.2.1	Concrete Security of RLWE	17
3.2.2	Concrete Security of NTRU	18
3.2.3	Concrete Security of FLIT	18
4	Performance Evaluation	19
4.1	Performance Analysis	19
4.2	Performance Test	20
4.2.1	Reference Implementation	20
4.2.2	AVX2 Optimized Implementation	20
4.2.3	Additional Implementation	20
4.2.4	Communication and Storage Overhead	21
5	Key Properties	21

1 Algorithm Specification

FLIT is an IND-CCA2-secure key-encapsulation mechanism (KEM) based on NTRU [12, 19], which takes advantage of repetition code to reduce decryption failure. The security is mainly based on the hardness of the module versions of the lattice problems LWE and NTRU.

Subsection 1.1 introduces the preliminaries and notation. Subsection 1.2 presents a novel technique called homomorphic encoding and decoding. The complete specification of FLIT is given in Subsection 1.3, followed by its correctness proof in Subsection 1.4. Finally, Subsection 1.5 summarizes the parameter sets proposed for different security levels.

1.1 Preliminaries and Notation

Bytes and byte arrays. Let $\mathcal{B} = \{0, 1, \dots, 255\}$ denote the set of 8-bit unsigned integers (bytes). We write $\mathcal{B}^k$ for the set of byte arrays of length k , and $\mathcal{B}^*$ for the set of all finite byte strings (or byte streams). When it is more convenient to work with bits rather than bytes, we explicitly convert using the function `BytesToBits`, which maps an array of ℓ bytes to an array of 8ℓ bits. For an input byte array $b_0, b_1, \dots, b_{\ell-1}$, the output bit β_i (with $0 \leq i < 8\ell$) is given by computing $\beta_i = \lfloor b_{\lfloor i/8 \rfloor} / 2^{i \bmod 8} \rfloor \bmod 2$.

Polynomial rings. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$, where N is a power of two. Depending on the security level, we use $(N, q) \in \{(512, 769), (1024, 769), (2048, 3329)\}$. Elements of $\mathcal{R}$ (resp. $\mathcal{R}_q$) are denoted by regular letters; this includes the images of $\mathbb{Z}$ (resp. $\mathbb{Z}_q$) under the natural embedding as constant polynomials. Bold lower-case letters denote vectors and bold upper-case letters denote matrices. For a polynomial $f \in \mathcal{R}$ or $\mathcal{R}_q$, we write $f[i]$ or f_i for the coefficient of X^i . We denote by $\mathcal{R}^\times$ the set of invertible elements in $\mathcal{R}$. Let $m = (m_0, m_1, \dots, m_{n-1}) \in \{0, 1\}^n$ be a message. We associate to it the polynomial

$$m(X) = m_0 + m_1X + \dots + m_{n-1}X^{n-1} \in \mathbb{Z}_2[X]/(X^n + 1).$$

Modular reduction. Let α be a positive integer. If α is even, we define $r' = r \bmod^\pm \alpha$ as the unique representative of $r \bmod \alpha$ lying in the interval $(-\frac{\alpha}{2}, \frac{\alpha}{2}]$. If α is odd, r' is taken as the unique element in $[-\frac{\alpha-1}{2}, \frac{\alpha-1}{2}]$ such that $r' \equiv r \pmod{\alpha}$. For the standard reduction, we write $r' = r \bmod^+ \alpha$ for the unique integer r' satisfying $0 \leq r' < \alpha$ and $r' \equiv r \pmod{\alpha}$. For $a \in \mathcal{R}_q$, we apply the reductions componentwise; e.g. $a \bmod^\pm \alpha$ denotes the polynomial whose i -th coefficient is $a_i \bmod^\pm \alpha$. When no ambiguity arises, we simply write $r \bmod \alpha$.

Norms. For an element $w \in \mathbb{Z}_q$, we define $|w|$ as $|w \bmod q|$. Let $w = w_0 + w_1X + \dots + w_{N-1}X^{N-1} \in \mathcal{R}$. We define the ℓ_∞ and ℓ_2 norms of w as

$$\|w\|_\infty = \max_{0 \leq i < N} |w_i|, \quad \|w\|_2 = \left(\sum_{i=0}^{N-1} |w_i|^2 \right)^{1/2}.$$

Sets and distributions. For a set S , the notation $s \leftarrow S$ denotes that s is sampled uniformly at random from S . If S instead represents a probability distribution, the same notation indicates that s is drawn according to S . We use $\mathcal{M}$ to denote the message space.

Compression and Decompression. We define two functions $\text{Compress}_q(x, d)$ and $\text{Decompress}_q(x, d)$, where $\text{Compress}_q(x, d)$ maps an element $x \in \mathbb{Z}_q$ to an integer in $\{0, \dots, 2^d - 1\}$ with $d < \lceil \log_2(q) \rceil$. These two functions satisfy

$$x' = \text{Decompress}_q(\text{Compress}_q(x, d), d),$$

where x' approximates x in the sense that

$$|x' - x \bmod^\pm q| \leq B_q := \left\lfloor \frac{q}{2^{d+1}} \right\rfloor.$$

Concretely, the two functions are defined as follows:

$$\begin{aligned} \text{Compress}_q(x, d) &= \left\lfloor \frac{2^d}{q} \cdot x \right\rfloor \bmod^+ 2^d, \\ \text{Decompress}_q(x, d) &= \left\lfloor \frac{q}{2^d} \cdot x \right\rfloor. \end{aligned}$$

The primary motivation for introducing Compress_q is to discard low-order bits in the ciphertext that contribute negligibly to the correctness of decryption, thereby reducing the ciphertext size.

Symmetric primitives. The design of FLIT makes use of a pseudorandom function $\text{PRF} : \mathcal{B}^{32} \times \mathcal{B} \rightarrow \mathcal{B}^*$, which is used to generate polynomials in $\mathcal{R}_q$. FLIT also employs two hash functions: $\text{H} : \mathcal{B}^* \rightarrow \mathcal{B}^{32}$ and $\text{G} : \mathcal{B}^* \rightarrow \mathcal{B}^{32} \times \mathcal{B}^{32}$ and a key-derivation function $\text{KDF} : \mathcal{B}^* \rightarrow \mathcal{B}^*$. For the 512-bit security level, the input and output lengths are increased to 64 bytes to ensure sufficient entropy.

Karatsuba Multiplication. *Karatsuba's algorithm* is a divide-and-conquer method for polynomial multiplication that recursively splits each operand into lower and upper halves, reducing the number of coefficient-level multiplications. In our setting, the Number Theoretic Transform (NTT) transforms polynomial multiplication in $\mathcal{R}_q$ into N/k independent base-case products of the form

$$c(X) = a(X) \cdot b(X) \bmod X^k - \zeta,$$

where $a(X), b(X) \in \mathbb{Z}_q[X]$, $\zeta \in \mathbb{Z}_q$, and $k \in \{4, 8, 16\}$ depending on the parameter set. Each such product is computed via Karatsuba's algorithm, which we describe starting from the base case $k = 2$.

Base case ($k = 2$). Write $a(X) = a_0 + a_1X$ and $b(X) = b_0 + b_1X$. The algorithm computes three auxiliary products

$$D_0 = a_0b_0, \quad D_1 = a_1b_1, \quad D_2 = (a_0 + a_1) \cdot (b_0 + b_1),$$

and recovers the two output coefficients of $c(X) = c_0 + c_1X \in \mathbb{Z}_q[X]/(X^2 - \zeta)$ as

$$c_0 = D_0 + \zeta D_1, \quad c_1 = D_2 - D_0 - D_1.$$

General case ($k = 2^m$, $m \geq 2$). For larger k , the algorithm is applied recursively. Given $a(X)$ and $b(X)$ of degree less than k , write

$$a(X) = a_0(X) + X^{k/2} \cdot a_1(X), \quad b(X) = b_0(X) + X^{k/2} \cdot b_1(X),$$

where $a_0, a_1, b_0, b_1 \in \mathbb{Z}_q[X]$ each have degree less than $k/2$. The algorithm recursively computes three half-size polynomial products

$$D_0 = a_0b_0, \quad D_1 = a_1b_1, \quad D_2 = (a_0 + a_1) \cdot (b_0 + b_1),$$

and assembles the full product as

$$c(X) = (D_0 + \zeta D_1) + X^{k/2} \cdot (D_2 - D_0 - D_1) \bmod X^k - \zeta.$$

The recursion bottoms out at $k = 2$, invoking the base case above.

Algorithm 1 $\text{KaratsubaMul}(a(X), b(X), \zeta, k)$

Input: $a(X), b(X) \in \mathbb{Z}_q[X]$ with $\deg < k$, $k = 2^m$

Output: $c(X) \equiv a(X) \cdot b(X) \bmod (X^k - \zeta)$

```

1: if  $k = 2$  then
2:    $D_0 := a_0 \cdot b_0$ 
3:    $D_1 := a_1 \cdot b_1$ 
4:    $D_2 := (a_0 + a_1) \cdot (b_0 + b_1)$ 
5:   return  $(D_0 + \zeta D_1) + X \cdot (D_2 - D_0 - D_1)$ 
6: end if
7: Split  $a = a_0 + X^{k/2} \cdot a_1$ ,  $b = b_0 + X^{k/2} \cdot b_1$  ▷ Decomposition for recursive Karatsuba
8:  $D_0 := \text{KaratsubaMul}(a_0, b_0, \zeta, k/2)$ 
9:  $D_1 := \text{KaratsubaMul}(a_1, b_1, \zeta, k/2)$ 
10:  $D_2 := \text{KaratsubaMul}(a_0 + a_1, b_0 + b_1, \zeta, k/2)$ 
11: return  $(D_0 + \zeta D_1) + X^{k/2} \cdot (D_2 - D_0 - D_1) \bmod (X^k - \zeta)$ 

```

NTTs, multiplication, and bit-reversed order. A very efficient way to perform multiplications in $\mathcal{R}_q$ is via the so-called *number-theoretic transform* (NTT), with the base-case multiplication further accelerated by Karatsuba’s algorithm.

For our prime $q = 769$ (resp. $q = 3329$) with $q - 1 = 2^8 \cdot 3$ (resp. $q - 1 = 2^8 \cdot 13$), the base field $\mathbb{Z}_q$ contains primitive 256-th roots of unity. Therefore, the defining polynomial $X^N + 1$ of $\mathcal{R}_q$ factors into 128 polynomials of degree $k = N/2^7 \in \{4, 8, 16\}$. Simple in-place implementations of the NTT without reordering outputs these polynomials in bit-reversed order and we define the NTT in this way. Let $\zeta \in \mathbb{Z}_q$ denote a fixed primitive 256-th root of unity and $\{\zeta, \zeta^3, \zeta^5, \dots, \zeta^{255}\}$ the set of all the 256-th roots of unity. The polynomial $X^N + 1$ can therefore be written as

$$X^N + 1 = \prod_{i=0}^{127} (X^k - \zeta^{2i+1}) = \prod_{i=0}^{127} (X^k - \zeta^{2\text{br}_7(i)+1}),$$

where $\text{br}_7(i)$ for $i = 0, \dots, 127$ denotes the bit-reversal of the unsigned 7-bit integer i . This particular ordering of the factors is useful for compatibility with the idiosyncrasies of AVX instructions. Then the NTT of $f \in \mathcal{R}_q$ is given by

$$(f \bmod X^k - \zeta^{2\text{br}_7(0)+1}, \dots, f \bmod X^k - \zeta^{2\text{br}_7(127)+1}).$$

This vector of linear polynomials is then serialized canonically into a vector in $\mathbb{Z}_q^N$. Moreover, in order to avoid introducing additional data types and to facilitate in-place implementations of the NTT, we define the NTT as a bijection $\text{NTT} : \mathcal{R}_q \rightarrow \mathcal{R}_q$ that maps $f \in \mathcal{R}_q$ to the polynomial with the aforementioned coefficient vector. Hence,

$$\text{NTT}(f) = \hat{f} = \hat{f}_0 + \hat{f}_1 X + \dots + \hat{f}_{N-1} X^{N-1}.$$

with coefficients given by

$$\hat{f}_{ki+r} = \sum_{j=0}^{127} f_{kj+r} \cdot \zeta^{(2\text{br}_7(i)+1)j},$$

where $r = 0, \dots, k - 1$ and $i = 0, \dots, 127$. We stress that although we represent $\hat{f}$ as a polynomial in $\mathcal{R}_q$, it does not carry a direct algebraic interpretation in this form. Instead, the natural representation of $\text{NTT}(f) = \hat{f}$ is as a collection of 128 polynomials of degree less than k , as described above.

Using the NTT and its inverse NTT^{-1} , we can efficiently compute the product of two polynomials $f, g \in \mathcal{R}_q$ via

$$f \cdot g = \text{NTT}^{-1}(\text{NTT}(f) \circ \text{NTT}(g)),$$

where $\circ$ denotes component-wise multiplication in the NTT domain. More precisely, writing $\hat{f} = (\hat{f}^{(0)}, \dots, \hat{f}^{(127)})$, $\hat{g} = (\hat{g}^{(0)}, \dots, \hat{g}^{(127)})$, with $\hat{f}^{(i)}, \hat{g}^{(i)} \in \mathbb{Z}_q[X]/(X^k - \zeta^{2\text{br}_7(i)+1})$, the component-wise product is given by

$$\hat{h}^{(i)} = \text{KaratsubaMul}(\hat{f}^{(i)}, \hat{g}^{(i)}, \zeta^{2\text{br}_7(i)+1}, k).$$

Throughout the document, we use NTT and NTT^{-1} to refer to the concrete instantiations defined above, and use the term NTT in a generic sense when referring to the general technique.

Sampling from a ternary distribution. All polynomials in FLIT are sampled from a ternary distribution $\mathcal{T}_\sigma$ with parameter $\sigma \in \{\frac{5}{32}, \frac{5}{16}, \frac{7}{16}, \frac{1}{8}, \frac{1}{4}\}$, defined by

$$\Pr[X = 1] = \Pr[X = -1] = \sigma, \quad \Pr[X = 0] = 1 - 2\sigma.$$

When we say that a polynomial $f \in \mathcal{R}_q$ is sampled from $\mathcal{T}_\sigma$, we mean that all coefficients are drawn independently according to this distribution. For the specification of FLIT, we additionally require a deterministic procedure to sample a polynomial $f \in \mathcal{R}_q$ from $\mathcal{T}_\sigma$ using a fixed-length byte array produced by a pseudorandom function. This is achieved via the function Ternary_σ .

The exact procedure depends slightly on the choice of σ . As a representative example, Algorithm 2 describes the case $\sigma = \frac{5}{32}$.

Algorithm 2 Ternary $_{\sigma}(B)$

Parameter: $\sigma = \frac{5}{32}$ **Input:** Byte array $B = (b_0, b_1, \dots, b_{5N/8-1}) \in \mathcal{B}^{\frac{5N}{8}}$ **Output:** $f \in \mathcal{R}_q$

```
1:  $(\beta_0, \beta_1, \dots, \beta_{5N-1}) := \text{BytesToBits}(B)$ 
2: for  $i$  from 0 to  $N-1$  do
3:    $v := \sum_{r=0}^4 \beta_{5i+r} \cdot 2^r$ 
4:   if  $v < 5$  then
5:      $f_i := 1$ 
6:   else if  $v < 10$  then
7:      $f_i := -1$ 
8:   else
9:      $f_i := 0$ 
10:  end if
11: end for
12: return  $f$ 
```

For different choices of σ , the number of random bits required to generate a single coefficient varies. Concretely, the construction uses 5 bits per coefficient for $\sigma = \frac{5}{32}$, 4 bits for $\sigma = \frac{5}{16}$ and $\sigma = \frac{7}{16}$, 3 bits for $\sigma = \frac{1}{8}$, and 2 bits for $\sigma = \frac{1}{4}$. Equivalently, the total input length for sampling a polynomial with N coefficients is given by the corresponding number of bytes listed in Table 1. These mappings partition the set of bit patterns into three regions realizing the probabilities $\Pr[X = 1]$, $\Pr[X = -1]$, and $\Pr[X = 0]$ without resorting to rejection sampling, which is beneficial for constant-time implementations.

Table 1: Sampling cost for $\mathcal{T}_{\sigma}$

σ	Bits / coeff.	Input size (N coeffs)
$\frac{5}{32}$	5	$\frac{5N}{8}$ bytes
$\frac{5}{16}$	4	$\frac{N}{2}$ bytes
$\frac{7}{16}$	4	$\frac{N}{2}$ bytes
$\frac{1}{8}$	3	$\frac{3N}{8}$ bytes
$\frac{1}{4}$	2	$\frac{N}{4}$ bytes

Inversion. In the decryption procedure, we require the inverse of the secret polynomial f in the ring $\mathbb{Z}_2[X]/(X^n + 1)$. We first determine whether f is invertible in $\mathbb{Z}_2[X]/(X^n + 1)$. A necessary and sufficient condition for invertibility is

$$\sum_{i=0}^{N-1} f[i] = 1 \pmod{2},$$

since n is a power of two and hence $X^n + 1 = (X + 1)^n$ over $\mathbb{Z}_2$. If this condition holds, we reduce f to $\mathbb{Z}_2[X]/(X^n + 1)$ via coefficient aggregation:

$$f_0[i] = \sum_{j=0}^{N/n-1} f[i + jn] \pmod{2}, \quad i = 0, \dots, n-1.$$

We then compute the inverse of f_0 in $\mathbb{Z}_2[X]/(X^n + 1)$. To facilitate Newton iteration, we apply the change of variables $X \mapsto X + 1$ and define $f_1(X) = f_0(X + 1)$, which ensures $f_1(0) = 1$, and thus guarantees the existence of a formal power-series inverse in $\mathbb{Z}_2[[X]]$. We compute f_1^{-1} using Newton iteration. Starting from $a = 1$, we iteratively refine $a \leftarrow f_1 \cdot a^2 \pmod{X^l}$, where the truncation degree l is doubled at each iteration until $l \geq n$. Finally, the inverse in $\mathbb{Z}_2[X]/(X^n + 1)$ is obtained by applying the inverse transform $a(X + 1)$. The complete procedure is summarized in Algorithm 3–5.

Algorithm 3 Inversion(f)

Input: $f \in \mathcal{R}_q$ **Output:** $f' = f_0^{-1} \in \mathbb{Z}_2[X]/(X^n + 1)$, or $\perp$

```
1: if  $\sum_{i=0}^{N/n-1} f[i] = 0 \pmod 2$  then
2:   return  $\perp$ 
3: end if
4: for  $i$  from 0 to  $n-1$  do
5:    $f_0[i] := \sum_{j=0}^{N/n-1} f[i + j \cdot n] \pmod 2$ 
6: end for
7:  $f_1 := \text{Translate}(f_0)$   $\triangleright f_1(X) = f_0(X + 1)$ 
8:  $a := \text{NewtonInv}(f_1)$ 
9: return  $\text{Translate}(a)$   $\triangleright f' = a(X + 1)$ 
```

Algorithm 4 NewtonInv(f_1)

Input: $f_1 \in \mathbb{Z}_2[X]$ with $\deg(f_1) < n$ and $f_1(0) = 1$ **Output:** $a \equiv f_1^{-1} \pmod{X^n}$ in $\mathbb{Z}_2[X]$

```
1:  $a := 1$ 
2: for  $l = 2, 4, 8, \dots, n$  do
3:    $a := f_1 \cdot a^2 \pmod{X^l}$   $\triangleright a \equiv f_1^{-1} \pmod{X^{l/2}} \Rightarrow f_1 a^2 \equiv f_1^{-1} \pmod{X^l}$ 
4: end for
5: return  $a$ 
```

Algorithm 5 Translate(f)

Input: $f \in \mathbb{Z}_2[X]/(X^n + 1)$ **Output:** $f(X + 1) \pmod{(X^n + 1)}$ in $\mathbb{Z}_2[X]$

```
1: for  $s = 1, 2, 4, \dots, n/2$  do
2:   for  $i = 0$  to  $n-1$  do
3:     if  $(i \& s) = 0$  then
4:        $f[i] := f[i] + f[i + s] \pmod 2$ 
5:     end if
6:   end for
7: end for
8: return  $f$ 
```

1.2 Homomorphic Encoding and Decoding.

We introduce *homomorphic Encoding and decoding* for $\mathcal{R}$, a novel technique for recovering the plaintext from an NTRU-style ciphertext. The core observation is that the repetition-based encoding used in FLIT interacts with polynomial multiplication in a structurally compatible way, as formalized below.

- **Encode:** the algorithm **Encode** maps message $m \in \{0, 1\}^n$ ($n \mid N$) to a polynomial in $\mathcal{R}$ (or $\mathcal{R}_q$ with the trivial embedding). Specifically, when taking a polynomial with binary coefficients as input, **Encode** acts on the binary coefficients.
- **Decode:** the algorithm **Decode** maps message a polynomial in $\mathcal{R}_q$ to $m \in \{0, 1\}^n$.

Property 1 (Homomorphism). For any polynomial $f \in \mathcal{R}$ and message $m \in \{0, 1\}^n$, the encoding algorithm **Encode**($\cdot$) satisfies

$$f \cdot \text{Encode}(m) \equiv \text{Encode}(f \cdot m(X) \pmod{(X^n + 1, 2)}) \pmod{(X^N + 1, 2)}.$$

It is surprising the repetition code satisfies the property, in which **Encode**(m) replicates each bit of m exactly N/n times across the N positions of $\mathcal{R}_q$. We present the encoding algorithm for $n = \frac{N}{2}$ and $n = \frac{N}{4}$ in Algorithm 6, which we need in our KEMs.

Algorithm 6 $\text{Encode}(m)$

Input: Message $m \in \{0, 1\}^n$ **Output:** Polynomial $M \in \mathcal{R}_q$

```
1: for  $i = 0$  to  $n - 1$  do
2:   if  $n = N/2$  then ▷ 128-bit security level
3:      $M[i] := m[i]$ 
4:      $M[i + n] := m[i]$ 
5:   else if  $n = N/4$  then ▷ 256-bit or 512-bit security level
6:      $M[i] := m[i]$ 
7:      $M[i + n] := m[i]$ 
8:      $M[i + 2n] := m[i]$ 
9:      $M[i + 3n] := m[i]$ 
10:  end if
11: end for
12: return  $M$ 
```

We claim that $\text{Encode}(\cdot)$ in Algorithm 6 satisfies Property 1.

Lemma 1. For any polynomial $f \in \mathcal{R}$ and message $m \in \{0, 1\}^n$, the encoding algorithm $\text{Encode}(\cdot)$ in Algorithm 6 satisfies

$$f \cdot \text{Encode}(m) \equiv \text{Encode}(f \cdot m(X) \bmod (X^n + 1, 2)) \bmod (X^N + 1, 2).$$

Proof. Note that the $\text{Encode}(m) = m(X) \cdot S(X)$, where $S(X) = 1 + X^n + X^{2n} + \dots + X^{N-n}$. For simplicity, we assume all polynomials are over $\mathbb{Z}_2$. It is enough to prove in $\mathbb{Z}_2[X]/(X^N + 1)$:

$$f(X) \cdot m(X) \cdot S(X) \equiv S(X) \cdot ((f(X) \cdot m(X)) \bmod (X^n + 1)).$$

Since $X^n + 1$ is monic, Euclidean division yields unique polynomials $q(X), r(X) \in \mathbb{Z}_2[X]$ such that

$$f(X)m(X) = q(X)(X^n + 1) + r(X), \quad \deg r(X) < n. \quad (1)$$

By construction,

$$r(X) \equiv (f(X)m(X)) \bmod (X^n + 1).$$

Note that in $\mathbb{Z}_2[X]$,

$$(X^n + 1)S(X) = (X^n + 1)(1 + X^n + X^{2n} + \dots + X^{N-n}) = X^N + 1.$$

Multiplying Equation (1) by $S(X)$ gives

$$f(X)m(X)S(X) = q(X)(X^n + 1)S(X) + r(X)S(X) = q(X)(X^N + 1) + r(X)S(X) \equiv r(X)S(X) \bmod (X^N + 1).$$

□

Property 1 then implies that $f \cdot \text{Encode}(m) \bmod (X^N + 1, 2)$ is itself a repetition-encoded polynomial. Notably, this homomorphic behavior holds for every choice of $f \in \mathcal{R}$, requiring no structural conditions on the secret polynomial.

In lattice-based cryptosystems, we usually use $\frac{q+1}{2}\text{Encode}(m)$ instead of $\text{Encode}(m)$ directly. In such setting, our Encode is exactly the same as the D_2 (or. D_4) lattice code, which is widely used in lattice-based cryptosystems [17, 3, 20]. Hence we can also employ the corresponding decoding algorithm to decode one plaintext bit from two (resp. four) coefficients of a ring element, thereby achieving a high tolerance against noise.

We present the Decode algorithm in Algorithm 7. Moreover, by [17, 20] we have the following lemma.

Lemma 2. When decoding $c = \frac{q+1}{2}\text{Encode}(m) + e \in \mathcal{R}_q$ where e is some error polynomial with coefficients denoted by e_i , $m \in \mathbb{Z}_2[X]/(X^n + 1)$ can be recovered correctly by Algorithm 7 if

$$\sum_{j=0}^{N/n-1} |e_{i+jn} \bmod \pm q| < \frac{N}{n} \cdot \frac{q-1}{4}.$$

Algorithm 7 Decode(c)

Input: Polynomial $c = \frac{q+1}{2} \text{Encode}(m) + e \in \mathcal{R}_q$, where e is some error polynomial

Output: Message $m \in \{0, 1\}^n$

```
1: for  $i = 0$  to  $n - 1$  do
2:   if  $n = N/2$  then ▷ 128-bit security level
3:     if  $\sum_{j=0}^1 |c[i + jn] - \frac{q+1}{2}| < \frac{q-1}{2}$  then
4:        $m[i] := 1$ 
5:     else
6:        $m[i] := 0$ 
7:     end if
8:   else if  $n = N/4$  then ▷ 256-bit or 512-bit security level
9:     if  $\sum_{j=0}^3 |c[i + jn] - \frac{q+1}{2}| < q - 1$  then
10:       $m[i] := 1$ 
11:    else
12:       $m[i] := 0$ 
13:    end if
14:   end if
15: end for
16: return  $m$ 
```

1.3 Specification

FLIT.CPAPKE is similar to the NTRU public-key encryption scheme, sharing the same polynomial ring $\mathbb{Z}_q[X]/(X^N + 1)$ and employing ternary secret distributions. However, FLIT introduces several modifications to improve security and performance, including the use of NTT-based multiplication, a repetition code for message encoding (i.e., each message bit is mapped to multiple coefficients of the polynomial), and an explicit compression step.

A detailed description of the parameters and algorithms follows.

Parameters. The FLIT.CPAPKE scheme is defined by the parameters $\lambda, N, n, q, \mathcal{T}_f, \mathcal{T}_g, \mathcal{T}_r, \mathcal{T}_e$ and d . More precisely,

- λ denotes the security parameter.
- We use the polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$, where N is some power of 2, q is the prime modulus.
- $\mathcal{R}_q^\times$ is the set of invertible elements in $\mathcal{R}_q$.
- n denotes the message length and $n \mid N$.
- $\mathcal{T}_f$ denotes the parameter for the ternary distribution of the coefficients of f .
- $\mathcal{T}_g$ denotes the parameter for the ternary distribution of the coefficients of g .
- $\mathcal{T}_r$ denotes the parameter for the ternary distribution of the coefficients of r .
- $\mathcal{T}_e$ denotes the parameter for the ternary distribution of the coefficients of e .
- d denotes the compression parameter for the ciphertext.

Using the notation introduced in Subsection 1.1, we describe the key generation, encryption, and decryption procedures of FLIT.CPAPKE in Algorithms 8, 9, and 10. For clarity, we additionally include brief comments to highlight the main steps of each algorithm.

1.3.1 Specification of FLIT.CPAPKE

Algorithm 8 FLIT.CPAPKE.KeyGen()

Input: Security parameter 1^λ

Output: Secret key sk , public key pk

```

1:  $z \leftarrow \{0, 1\}^\lambda$ 
2:  $\kappa := 0$ 
3: repeat
4:    $f := \text{Ternary}_f(\text{PRF}(z, \kappa))$  ▷ Sample  $f \in \mathcal{R}_q$  from  $\text{Ternary}_{\mathcal{T}_f}$ 
5:    $\kappa := \kappa + 1$ 
6: until  $f \in \mathcal{R}_2^\times$  and  $f \in \mathcal{R}_q^\times$ 
7:    $g := \text{Ternary}_g(\text{PRF}(z, \kappa))$  ▷ Sample  $g \in \mathcal{R}_q$  from  $\text{Ternary}_{\mathcal{T}_g}$ 
8:    $f' := \text{Inversion}(f)$  ▷  $f'$  is the inverse of  $f$  in  $\mathbb{Z}_2[X]/(X^n + 1)$ 
9:    $\hat{f} := \text{NTT}(f)$ 
10:   $\hat{g} := \text{NTT}(g)$ 
11:   $\hat{h} := \hat{g} \circ \hat{f}^{-1}$ 
12:   $sk := (\hat{f}, f')$ 
13:   $pk := \hat{h}$  ▷  $pk := \hat{h}$  in NTT domain
14: return  $(sk, pk)$ 

```

Algorithm 9 FLIT.CPAPKE.Enc(pk, m, ρ)

Input: Public key $pk = \hat{h}$

Input: Message m

Input: Random coins ρ

Output: Ciphertext c

```

1:  $r := \text{Ternary}_r(\text{PRF}(\rho, 0))$  ▷ Sample  $r \in \mathcal{R}_q$  from  $\text{Ternary}_{\mathcal{T}_r}$ 
2:  $e := \text{Ternary}_e(\text{PRF}(\rho, 1))$  ▷ Sample  $e \in \mathcal{R}_q$  from  $\text{Ternary}_{\mathcal{T}_e}$ 
3:  $\hat{r} := \text{NTT}(r)$ 
4:  $c_1 := \text{NTT}^{-1}(\hat{h} \circ \hat{r}) + e + \frac{q+1}{2} \cdot \text{Encode}(m)$  ▷  $c_1 = hr + e + \frac{q+1}{2} \cdot \text{Encode}(m)$ 
5:  $c := \text{Compress}_q(c_1, d)$ 
6: return  $c$ 

```

Algorithm 10 FLIT.CPAPKE.Dec(sk, c)

Input: Ciphertext c

Input: Secret key $sk = (\hat{f}, f')$

Output: Message m

```

1:  $c_1 := \text{Decompress}_q(c, d)$ 
2:  $\hat{c}_1 := \text{NTT}(c_1)$ 
3:  $c_2 := \text{NTT}^{-1}(\hat{f} \circ \hat{c}_1)$  ▷  $c_2 = gr + fe + \frac{q+1}{2} f \cdot \text{Encode}(m)$ 
4:  $m' := \text{Decode}(c_2)$  ▷  $m' = f \cdot m \bmod (X^n + 1, 2)$ 
5:  $m := m' \cdot f' \bmod (X^n + 1, 2)$  ▷  $f'$  is the inverse of  $f$  in  $\mathbb{Z}_2[X]/(X^n + 1)$ 
6: return  $m$ 

```

1.3.2 Specification of FLIT.CCAKEM

We construct FLIT as an IND-CCA-secure KEM by applying a slightly modified Fujisaki–Okamoto transform [10] to the IND-CPA-secure public-key encryption scheme introduced in the previous section. The key generation, encapsulation, and decapsulation procedures are formally described in Algorithms 11, 12, and 13, respectively.

Algorithm 11 FLIT.CCAKEM.KeyGen()

Input: Security parameter 1^λ **Output:** Secret key sk **Output:** Public key pk

- 1: $\rho \leftarrow \{0, 1\}^\lambda$
 - 2: $(sk', pk) = \text{FLIT.CPAPKE.KeyGen}()$
 - 3: $sk := (sk' || pk || H(pk) || \rho)$
 - 4: **return** (sk, pk)
-

Algorithm 12 FLIT.CCAKEM.Enc(pk)

Input: Public key pk **Output:** Ciphertext c **Output:** Shared key K

- 1: $m \leftarrow \{0, 1\}^\lambda$
 - 2: $m := H(m)$ ▷ Do not send output of system RNG
 - 3: $(\bar{K}, r) := G(m || H(pk))$
 - 4: $c := \text{FLIT.CPAPKE.Enc}(pk, m, r)$
 - 5: $K := \text{KDF}(\bar{K} || H(c))$
 - 6: **return** (c, K)
-

Algorithm 13 FLIT.CCAKEM.Dec(sk, c)

Input: Ciphertext c **Input:** Secret key sk **Output:** Shared key K

- 1: $m' = \text{FLIT.CPAPKE.Dec}(sk, c)$
 - 2: $(\bar{K}', r') := G(m' || H(pk))$
 - 3: $c' := \text{FLIT.CPAPKE.Enc}(pk, m', r')$
 - 4: **if** $c = c'$ **then**
 - 5: **return** $K := \text{KDF}(\bar{K}', H(c))$
 - 6: **else**
 - 7: **return** $K := \text{KDF}(\rho, H(c))$
 - 8: **end if**
-

1.4 Correctness

Theorem 1. In FLIT, for any message $m \in \{0, 1\}^n$, let $M = \text{Encode}(m) \in \mathcal{R}_q$, e' be the error polynomial introduced by compression, and $u = gr + f(e + e') + \left\lfloor \frac{fM}{2} \right\rfloor$. The decryption succeeds if for every $i \in \{0, 1, \dots, n-1\}$,

$$\sum_{j=0}^{N/n-1} |u_{i+jn \bmod \pm q}| < \frac{N}{n} \cdot \frac{q-1}{4}.$$

Proof. By the decryption,

$$c_2 = f \cdot c_1 = f \cdot (hr + e + \frac{q+1}{2} \text{Encode}(m) + e') = gr + f(e + e') + \frac{q+1}{2} fM.$$

Since only $f \cdot M \bmod 2$ is relevant for decryption, the remaining part of $f \cdot M$ can be treated as an additional error term. We denote by $\left\lfloor \frac{fM}{2} \right\rfloor$ the polynomial obtained by dividing each coefficient of fM

by 2 and taking the floor. Then we have

$$\begin{aligned}
c_2 &= gr + f(e + e') + \frac{q+1}{2}fM \\
&= gr + f(e + e') + \left\lfloor \frac{fM}{2} \right\rfloor + \frac{q+1}{2}(fM \bmod 2) \\
&= u + \frac{q+1}{2}(fM \bmod 2).
\end{aligned}$$

By Lemma 1, we have

$$fM \bmod 2 = \text{Encode}(f \cdot m \bmod (X^n + 1, 2)).$$

Then

$$c_2 = u + \frac{q+1}{2} \text{Encode}(f \cdot m \bmod (X^n + 1, 2)).$$

By Lemma 2, we know that if for every $i \in \{0, 1, \dots, n-1\}$,

$$\sum_{j=0}^{N/n-1} |u_{i+jn} \bmod \pm q| < \frac{N}{n} \cdot \frac{q-1}{4},$$

we can recover $f \cdot m \bmod (X^n + 1, 2)$ correctly, and then multiplying it by f' can yield the correct m . $\square$

Decryption failure probability. The above theorem provides the condition for successful decryption. The concrete failure probability δ is computed numerically with the help of the `FLIT.py` Python script which is available online at <https://gitcode.com/000toBomb/FLIT>. It is adapted from Kyber¹ [5]. The computation relies on the independence assumption among the coefficients. Under this assumption, each coefficient of the total error term u is independent, with its distribution derived from g, r, f, e, e' , and the message (which we treat as uniformly distributed). The overall failure probability δ is then obtained by computing the tail probability of the sum in the success condition and applying the union bound over the n positions.

1.5 FLIT Parameter Sets

We define three parameter sets for FLIT, referred to as FLIT128, FLIT256, and FLIT512, with the corresponding parameters listed in Table 2. The table additionally includes the derived parameter δ , representing the decryption failure probability on a valid FLIT ciphertext in the average case. The parameter choices are motivated by the following considerations.

Table 2: Parameter sets for FLIT.

Scheme	N	n	q	$\mathcal{T}_f$	$\mathcal{T}_g$	$\mathcal{T}_r$	$\mathcal{T}_e$	d	δ
FLIT128	512	256	769	$\frac{1}{8}$	$\frac{5}{16}$	$\frac{1}{8}$	$\frac{5}{16}$	8	$2^{-187.5}$
FLIT256	1024	256	769	$\frac{5}{32}$	$\frac{1}{4}$	$\frac{5}{32}$	$\frac{1}{4}$	8	$2^{-176.1}$
FLIT512	2048	512	3329	$\frac{7}{16}$	$\frac{7}{16}$	$\frac{7}{16}$	$\frac{7}{16}$	9	$2^{-195.5}$

- The parameter n is chosen to ensure that the encapsulated key has sufficient entropy, as it determines the size of the message space. N is set to 512, 1024, and 2048, corresponding to security levels of 128, 256, and 512 bits respectively (see Table 3 in §3.2.3 for concrete security). Larger N enhances both security and error resilience due to the increased repetition N/n .
- We choose q to be a small prime such that $\frac{N}{\ell} \mid (q-1)$ for some integer ℓ , which ensures the existence of an efficient NTT-based multiplication. To reduce the sizes of the public key and ciphertext, we select q to be as small as possible while maintaining the desired security level and an acceptable decryption failure probability. Thus we choose $q = 769$ for 128-bit and 256-bit security, and $q = 3329$ for 512-bit security.
- The remaining parameters $\mathcal{T}_f, \mathcal{T}_g, \mathcal{T}_r, \mathcal{T}_e$ and d are chosen to balance security, ciphertext size, and decryption failure probability.

¹<https://github.com/pq-crystals/security-estimates>

Instantiating H, G, PRF and KDF. To complete the specification of FLIT, it remains to instantiate the symmetric primitives. All primitives are instantiated using the SM3-based hash function and XOF-like primitives provided with the submission API package:

- H is instantiated with `sm3hash` in FLIT128 and FLIT256, and with `pseudohash` in FLIT512;
- G is instantiated with `pseudohash`;
- PRF and KDF are instantiated with `pseudoXOF`.

2 Design Rationale

The security of FLIT relies on the hardness of the RLWE assumption (Def. 1) and the NTRU assumption (Def. 2). The IND-CPA security of FLIT.CPAPKE is directly based on these two assumptions, and the IND-CCA2 security of FLIT.CCAKEM is then obtained via the Fujisaki–Okamoto (FO) transform.

The construction follows the NTRU variant of Stehlé and Steinfeld [19]. The main difference from the standard construction is that we repetitively encode the message m into the higher-order bits of the ciphertext polynomial, and recover it via homomorphic decoding. This repetition significantly reduces the decryption failure probability and, in turn, allows the use of compression techniques to reduce the ciphertext size.

Parameter selection rationale. The ring dimension N is chosen as a power of two to support efficient NTT-based polynomial multiplication. The three values $N = 512, 1024$, and 2048 correspond to the three target security levels, with larger N providing stronger lattice security and greater error tolerance via the increased repetition factor N/n . The inner dimension n determines the message length and, consequently, the entropy of the encapsulated key. We set $n = 256$ for FLIT128 and FLIT256, and $n = 512$ for FLIT512, ensuring that the message space is large enough to resist brute-force attacks on the shared secret.

The modulus q is selected to satisfy two competing requirements. On the one hand, q must be large enough to accommodate the noise introduced by the secret and error polynomials without causing excessive decryption failures. On the other hand, a smaller q reduces the sizes of the public key and ciphertext and strengthens security against lattice reduction attacks. We select $q = 769$ for FLIT128 and FLIT256, and $q = 3329$ for FLIT512, where the larger $N = 2048$ and the more aggressive noise parameters demand a larger modulus. In all cases, q is chosen such that $\frac{N}{\ell} \mid (q - 1)$ for some integer ℓ , which enables efficient NTT-based polynomial multiplication over $\mathbb{Z}_q$.

The noise distributions $\mathcal{T}_f$, $\mathcal{T}_g$, $\mathcal{T}_r$, and $\mathcal{T}_e$ determine the fraction of nonzero coefficients in the corresponding polynomials. For FLIT128 and FLIT256, we deliberately align the parameters so that the underlying RLWE and NTRU instances have comparable hardness. Within these two levels, $\mathcal{T}_f$ is chosen to be smaller than $\mathcal{T}_g$ because f has a larger impact on the decryption failure probability. The specific values are selected to balance security and the decryption failure probability. For FLIT512, we set all four distributions to $7/16$, a more aggressive choice made possible by the larger modulus $q = 3329$, which provides greater tolerance against decryption errors.

The compression parameter d is chosen to reduce the size of ciphertext. A smaller d reduces the ciphertext size but introduces more compression error e' , which raises the decryption failure probability δ . We therefore choose d to be as small as possible while keeping δ below the target bound for each security level. This yields $d = 8$ for FLIT128 and FLIT256, and $d = 9$ for FLIT512.

Analysis of decryption failure probability. The values of δ reported in Table 2 are computed under the average case, where the message m is uniformly random. The resulting failure probabilities are $2^{-187.5}$ for FLIT128, $2^{-176.1}$ for FLIT256, and $2^{-195.5}$ for FLIT512. For a complete assessment, we also consider the worst-case message, i.e., m is the all-1s string, since the error term introduced by the message depends on m and is maximized in this case. The worst-case decryption failure probabilities are $2^{-177.8}$, $2^{-164.5}$, and $2^{-191.0}$, respectively.² With at most 2^{80} decryption queries, an adversary cannot observe a single decryption failure with overwhelming probability in all cases. This renders decryption failure attacks [8] infeasible.

Homomorphic encoding and decoding. The correctness of decryption depends on the homomorphic property $f \cdot \text{Encode}(m) \equiv \text{Encode}((f \cdot m(X)) \bmod (X^n + 1, 2)) \pmod{2}$; hence we refer to our decoding technique as homomorphic encoding and decoding.

²These values are computed in the FLIT.py script

Compression Technique. We apply two compression techniques to polynomial coefficients. First, for the ciphertext, each coefficient is truncated by dropping its low-order bits, reducing the per-coefficient width from $\lceil \log_2 q \rceil$ to d bits ($d = 8$ for FLIT128/FLIT256, $d = 9$ for FLIT512). Second, for the $q = 769$ parameter sets (FLIT128 and FLIT256), we follow [9], exploiting the gap between q and the next power of two by jointly encoding groups of 5 coefficients as a single base-97 integer. Concretely, each coefficient is split into a 7-bit high part and a 3-bit low part. The 5 low parts are packed directly into 15 bits, while the 5 high parts are interpreted as a 5-digit base-97 number and stored in 33 bits, requiring only 48 bits total for the group instead of the 50 bits of a naive direct packing. For $q = 3329$ (FLIT512), we do not exploit this gap.

Allow decapsulation failures. We design FLIT to allow decapsulation failures (i.e., decryption failures in FLIT.CPAPKE). Compared to schemes with zero failure probability, this approach can increase security against attacks that target the underlying lattice problem and also improve performance. Although non-negligible decapsulation failure probabilities are generally problematic, the extremely low failure rate in FLIT makes such attacks [14] much less threatening.

Additional Hashes. In the CCA transform, we hash the (hash of the) public key pk into the pre-key $\bar{K}$ and into the random coins r (see line 3 of Alg. 12), and we hash the (hash of the) ciphertext into the final key K . These hashes are not strictly required for the security reduction, but they improve robustness. Specifically, the final shared key output by the FLIT.CCAKEM depends on the full view of the exchanged messages, i.e., both the public key and the ciphertext. This makes the KEM contributory and safe to use in authenticated key exchanges without additional context hashing. Moreover, hashing pk into the random coins r provides additional protection against a class of multi-target attacks that exploit protocol failures.

Return value for decapsulation failure. In the event of decapsulation failure, the traditional FO transform returns $\perp$ (i.e., a distinguished failure symbol). We adopt a variant in which, upon failure, the resulting shared key is instead set to a pseudorandom value derived from the hash of a secret ρ and the ciphertext c . This variant was proven secure in [13]. In practice, it ensures that FLIT’s decapsulation implementations remain safe even if higher-level protocols neglect to check the return value. Owing to this implicit rejection mechanism, our decapsulation implementations always return 0 (indicating success).

3 Security Analysis

This section presents the security analysis of FLIT, covering both theoretical and concrete security aspects. The analysis proceeds as follows. In theoretical security, we first define the underlying hardness assumptions. Second, we introduce the security definition considered in this work. Third, we present the security reduction. For concrete security, we first analyze the main attack methods against RLWE and NTRU assumption, and then estimate the classical and quantum security strength of FLIT based on this analysis.

3.1 Theoretical Security

3.1.1 Hardness Assumption

We begin by recalling the computational hardness assumptions underlying the security of our construction. In particular, our scheme relies on the hardness of distinguishing structured ring elements from uniformly random ones. We formalize this via the decisional variants of the RLWE and NTRU problems.

Definition 1 (Decision-RLWE $_{n,q,\chi_r,\chi_e}$). For positive integers n, q and distribution χ_r, χ_e over $\mathcal{R}_q$, we define the advantage of an adversary $\mathcal{A}$ in solving the *Decision-RLWE* $_{n,q,\chi_r,\chi_e}$ problem as:

$$\text{Adv}_{n,q,\chi_r,\chi_e}^{\text{RLWE}}(\mathcal{A}) = \left| \frac{\Pr[\mathcal{A}(a, b) = 1 \mid a \leftarrow \mathcal{R}_q; b \leftarrow \mathcal{R}_q]}{-\Pr[\mathcal{A}(a, b = ar + e) = 1 \mid a \leftarrow \mathcal{R}_q; r \leftarrow \chi_r; e \leftarrow \chi_e]} \right|$$

Definition 2 (Decision-NTRU $_{n,q,\chi_f,\chi_g}$). For positive integers n, q and distribution χ_f, χ_g over $\mathcal{R}_q$, we define the advantage of an adversary $\mathcal{A}$ in solving the *Decision-NTRU* $_{n,q,\chi_f,\chi_g}$ problem as:

$$\text{Adv}_{n,q,\chi_f,\chi_g}^{\text{NTRU}}(\mathcal{A}) = \left| \frac{\Pr[\mathcal{A}(u) = 1 \mid u \leftarrow \mathcal{R}_q]}{-\Pr[\mathcal{A}(h = g \cdot f^{-1}) = 1 \mid f \leftarrow \chi_f; g \leftarrow \chi_g]} \right|$$

Game IND-CPA:	Game IND-CCA:	
1. $(pk, sk) \leftarrow \Pi_{\text{PKE}}.\text{KeyGen}(1^\lambda)$	1. $(pk, sk) \leftarrow \Pi_{\text{KEM}}.\text{KeyGen}(1^\lambda)$	
2. $(m_0, m_1) \leftarrow \mathcal{A}(pk)$	2. $b \xleftarrow{\$} \{0, 1\}$	Oracle $\mathcal{O}_{\text{Decaps}}(c)$:
3. $b \xleftarrow{\$} \{0, 1\}$	3. $(c^*, k_0^*) \leftarrow \Pi.\text{Encaps}(pk)$	1. if $c = c^*$ then return $\perp$
4. $c^* \leftarrow \Pi.\text{Enc}(pk, m_b)$	4. $k_1^* \leftarrow \mathcal{K}$	2. $k \leftarrow \text{Decaps}(sk, c)$
5. $b' \leftarrow \mathcal{A}(c^*, pk)$	5. $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Decaps}}}(k_b^*, c^*, pk)$	3. return k
6. return $b' = b$	6. return $b' = b$	

Figure 1: IND-CPA and IND-CCA security games.

Both assumptions state that, for appropriate parameter choices, no probabilistic polynomial-time adversary can distinguish the corresponding structured distributions from uniform with non-negligible advantage.

3.1.2 Security Definition

We now formalize the security notions considered in this work. We adopt the standard game-based definitions of indistinguishability for public-key encryption and key-encapsulation mechanisms.

Definition 3 (IND-CPA Security for PKE). A public-key encryption scheme Π_{PKE} is *indistinguishable under chosen-plaintext attack (IND-CPA)* secure if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{IND-CPA}}(\kappa) := \left| \Pr[\mathcal{A} \text{ outputs } \mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CPA security game (see Fig. 1) is negligible in the security parameter λ .

Correctness. The PKE scheme is said to be δ -correct if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr [\text{Dec}(sk, \text{Enc}(pk, m)) \neq m] \right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and the randomness of encryption.

Definition 4 (IND-CCA Security for KEM). A key-encapsulation mechanism Π_{KEM} is *indistinguishable under chosen-ciphertext attack (IND-CCA)* secure if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, its advantage

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{IND-CCA}}(\kappa) := \left| \Pr[\mathcal{A} \text{ outputs } \mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CCA security game (see Fig. 1) is negligible in the security parameter λ .

Correctness. The KEM scheme is δ -correct if

$$\mathbb{E} \left[\Pr [\text{Decaps}(sk, c) \neq k \mid (c, k) \leftarrow \text{Encaps}(pk, 1^\lambda)] \right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ and the randomness of encapsulation.

3.1.3 Security Reduction

We first note that FLIT.CPAPKE is tightly secure under the RLWE hardness assumption and NTRU hardness assumption.

Theorem 2. Let $\mathcal{A}$ be an IND-CPA adversary against the FLIT.CPAPKE scheme described in Algorithms 8, 9, and 10. Then there exist adversaries $\mathcal{B}$ against the decisional NTRU assumption and $\mathcal{C}$ against the decisional Ring-LWE assumption, both with roughly the same running time as $\mathcal{A}$, such that

$$\text{Adv}_{\text{FLIT.CPAPKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{n, q, \chi_f, \chi_g}^{\text{NTRU}}(\mathcal{B}) + \text{Adv}_{n, q, \chi_r, \chi_e}^{\text{RLWE}}(\mathcal{C}).$$

Proof. The adversary $\mathcal{A}$ is given access to the public key pk and may issue encryption queries for the FLIT.CPAPKE scheme. Afterwards, $\mathcal{A}$ selects two messages m_0 and m_1 . The challenger then randomly chooses one of them and encrypts it. The goal of the adversary is to determine whether the ciphertext corresponds to m_0 or m_1 .

To prove Theorem 2, we consider a sequence of games Game_0 to Game_2 , where Game_0 is the standard IND-CPA game, and Game_2 corresponds to a random instance. The security of the scheme follows by showing that Game_0 and Game_2 are computationally indistinguishable from the perspective of $\mathcal{A}$. We denote by $\text{Adv}_{\text{Game}_i}^{\text{IND-CPA}}(\mathcal{A})$ the advantage of $\mathcal{A}$ in Game_i , where $i \in \{0, 1, 2\}$.

Game₀. This is the standard IND-CPA security game for the FLIT.CPAPKE scheme.

- **Public key:** The public key h is generated according to Algorithm 8, where $h = gf^{-1} \in \mathcal{R}_q$.
- **Encryption query:** For any message m submitted by adversary $\mathcal{A}$, encryption is performed following Algorithm 9.
- **Challenge:** Given two messages m_0, m_1 chosen by $\mathcal{A}$, Game_0 samples $b \in \{0, 1\}$ uniformly at random and computes the ciphertext $c_b = \text{FLIT.CPAPKE.Enc}(pk, m_b, \rho)$, where ρ is determined by the challenger, and sends c_b to $\mathcal{A}$. The adversary outputs a guess $b' \in \{0, 1\}$ and wins if $b' = b$.

By definition, we have

$$\text{Adv}_{\text{Game}_0}^{\text{IND-CPA}}(\mathcal{A}) = \text{Adv}_{\text{FLIT.CPAPKE}}^{\text{IND-CPA}}(\mathcal{A}).$$

Game₁. Game_1 is identical to Game_0 except for how the public key is generated.

- **Public key:** In this game, h is sampled uniformly from $\mathcal{R}_q$ instead of being computed as $h = gf^{-1}$.

The only difference between Game_0 and Game_1 lies in the distribution of h : in Game_0 , h is an NTRU instance $h = gf^{-1}$, whereas in Game_1 , it is uniform. Hence, any advantage of $\mathcal{A}$ in distinguishing these two games is bounded by the decisional NTRU assumption:

$$\left| \text{Adv}_{\text{Game}_0}^{\text{IND-CPA}}(\mathcal{A}) - \text{Adv}_{\text{Game}_1}^{\text{IND-CPA}}(\mathcal{A}) \right| \leq \text{Adv}_{n,q,\chi_f,\chi_g}^{\text{NTRU}}(\mathcal{B}).$$

Game₂. Game_2 is identical to Game_1 except for the generation of the challenge ciphertext.

- **Challenge:** For the challenge messages m_0, m_1 , Game_2 samples $b \in \{0, 1\}$ uniformly at random and draws $u \leftarrow \mathcal{R}_q$. It then computes

$$c_b = \text{Compress}\left(u + \frac{q+1}{2} \cdot \text{Encode}(m_b), d\right)$$

instead of using the term $hr + e$.

The difference between Game_1 and Game_2 is that in the former, the masking term $hr + e$ follows a Ring-LWE distribution, whereas in the latter, u is uniformly random. Therefore, any distinguishing advantage of $\mathcal{A}$ is bounded by the decisional Ring-LWE assumption:

$$\left| \text{Adv}_{\text{Game}_1}^{\text{IND-CPA}}(\mathcal{A}) - \text{Adv}_{\text{Game}_2}^{\text{IND-CPA}}(\mathcal{A}) \right| \leq \text{Adv}_{n,q,\chi_r,\chi_e}^{\text{RLWE}}(\mathcal{C}).$$

Finally, in Game_2 , since u is uniformly distributed over $\mathcal{R}_q$, the value $u + \frac{q+1}{2} \cdot \text{Encode}(m_b)$ is also uniformly distributed, independently of b . As a result, the distributions corresponding to m_0 and m_1 are computationally indistinguishable to $\mathcal{A}$, implying that $\text{Adv}_{\text{Game}_2}^{\text{IND-CPA}}(\mathcal{A})$ is negligible.

Combining the above results, we obtain:

$$\text{Adv}_{\text{FLIT.CPAPKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{n,q,\chi_f,\chi_g}^{\text{NTRU}}(\mathcal{B}) + \text{Adv}_{n,q,\chi_r,\chi_e}^{\text{RLWE}}(\mathcal{C}).$$

This completes the proof of Theorem 2. $\square$

As for security in the quantum random oracle model (QROM), [15, 18] proved that FLIT.CCAKEM is IND-CCA secure in the QROM, provided that FLIT.CPAPKE is IND-CPA secure.

Theorem 3. FLIT.CPAPKE is δ -correct with message space $\mathcal{M}$. Let $\mathcal{A}$ be an IND-CCA adversary against the FLIT.CCAKEM scheme described in Algorithms 11, 12, and 13. Assume that $\mathcal{A}$ makes at most q_D queries to the decapsulation oracle and q (quantum) queries to the random oracles H and G . Then there exists an IND-CPA adversary $\mathcal{B}$ against FLIT.CPAPKE, described in Algorithms 8, 9, and 10, such that

$$\text{Adv}_{\text{FLIT.CCAKEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2q\sqrt{\text{Adv}_{\text{FLIT.CPAPKE}}^{\text{IND-CPA}}(\mathcal{B})} + \frac{2q}{\sqrt{|\mathcal{M}|}} + 4q\sqrt{\delta}.$$

Moreover, the running time of $\mathcal{B}$ is essentially the same as that of $\mathcal{A}$.

3.2 Concrete Security

A KEM necessitates rigorous security analysis for both the private key and the shared key. The security of the private key is anchored in the decisional NTRU assumption, while that of the shared key derives from the decisional RLWE assumption. This section presents methodologies for analyzing both, followed by concrete security evaluations using state-of-the-art estimation techniques. We define **Vec** mapping polynomial $a = \sum_{i=0}^{n-1} a_i x^i$ to row vector $(a_i)_{0 \leq i < n}$, and **Mat** mapping $a \in \mathcal{R}_q$ to matrix $(\text{Vec}(x^i a))_{0 \leq i < n}$.

3.2.1 Concrete Security of RLWE

To assess the concrete security of RLWE, one considers the complexity of the most efficient known attacks on the problem. These include primal and dual lattice attacks, as well as combinatorial and algebraic approaches. Each of these attacks targets different aspects of the RLWE structure, and their costs provide the basis for parameter selection. We define an RLWE instance as $(a, b = ar + e)$, where $a \leftarrow \mathcal{R}_q$, $r \leftarrow \chi_r$, and $e \leftarrow \chi_e$.

Primal Attack. The primal attack reduces an RLWE instance to a unique Shortest Vector Problem (uSVP). We construct a $(2n + 1)$ -dimensional embedding lattice $\Lambda \subset \mathbb{Z}^{2n+1}$ defined by

$$\Lambda = \{(\mathbf{x}, \mathbf{y}, t) \in \mathbb{Z}^n \times \mathbb{Z}^n \times \mathbb{Z} \mid \mathbf{x} \equiv \mathbf{y} \cdot \text{Mat}(a) + t \cdot \text{Vec}(b) \pmod{q}\}.$$

The goal of the attacker is to recover the target short vector $\mathbf{v} = (\mathbf{e}, \mathbf{r}, -1) \in \Lambda$. Since $\|\mathbf{v}\|$ is expected to be significantly smaller than that of other non-parallel lattice vectors, the problem can be solved by applying the BKZ [6] algorithm to recover this unique shortest vector.

Dual Attack. In the dual attack, $\text{Mat}(a)$ is partitioned into $\mathbf{A}_0 \in \mathbb{Z}_q^{n_0 \times n}$ and $\mathbf{A}_1 \in \mathbb{Z}_q^{n_1 \times n}$, while $\text{Vec}(r)$ is decomposed into $\mathbf{r}_0 \in \mathbb{Z}^{n_0}$ and $\mathbf{r}_1 \in \mathbb{Z}^{n_1}$. We then construct the lattice

$$\Lambda = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^{n_0} \times \mathbb{Z}^n \mid \mathbf{x} \equiv \mathbf{y} \cdot \mathbf{A}_0^T \pmod{q}\}.$$

Applying BKZ to this lattice yields a set of short vectors v_i satisfying $v_i \mathbf{A}_0^T \equiv w_i \pmod{q}$, where both $\|v_i\|$ and $\|w_i\|$ remain small. This gives rise to the relation

$$\langle v_i, \text{Vec}(b) \rangle \equiv \langle v_i \mathbf{A}_1^T, \mathbf{r}_1 \rangle + \langle w_i, \mathbf{r}_0 \rangle + \langle v_i, \text{Vec}(e) \rangle \pmod{q}.$$

Since the term $\langle w_i, \mathbf{r}_0 \rangle + \langle v_i, \text{Vec}(e) \rangle$ is of small magnitude, the correct value of $\mathbf{r}_1$ can be recovered via enumeration. The overall computational cost of this attack, including both lattice reduction and enumeration, can be estimated using the methods of [11, 16].

Combinatorial and Algebraic Attacks. The Blum–Kalai–Wasserman (BKW) algorithm represents the primary combinatorial approach to attacking RLWE. While structurally similar to the dual attack, it replaces lattice reduction with combinatorial operations and can be effective for certain parameter choices; its complexity can be estimated following [1].

Algebraic attacks [4] exploit the small support of the error terms $e[i]$ to derive a system of n high-degree polynomial equations in n unknowns $r[i]$, which are solved using Gröbner basis algorithms. Their complexity is dominated by Gröbner basis computation, and they are generally not competitive in practice for typical parameter settings.

3.2.2 Concrete Security of NTRU

We define an NTRU instance as $h = gf^{-1}$, where $g \leftarrow \chi_g \cap \mathcal{R}_q^\times$ and $f \leftarrow \chi_f \cap \mathcal{R}_q^\times$. As in the Ring-LWE setting, the ring structure of NTRU can be represented in matrix form, where each polynomial corresponds to a matrix in $\mathbb{Z}^{n \times n}$. In particular, we consider the lattice

$$\Lambda = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^n \times \mathbb{Z}^n \mid \mathbf{x} \equiv \mathbf{y} \cdot \mathbf{Mat}(h) \pmod{q}\}.$$

Moreover, the NTRU instance can be written as $hf - g = 0$, which can be viewed as a special case of the Ring-LWE instance with $b = 0$. As a result, most attacks against Ring-LWE can be adapted to the NTRU setting, with the notable exception of the dual attack.

3.2.3 Concrete Security of FLIT

Building on our analysis, we use the Lattice Estimator [2] (commit 6019056) to derive concrete security parameters for FLIT. This tool provides security estimates for both RLWE and NTRU instantiations, as summarized in Table 3. For each attack strategy, we compute its computational cost and take the minimum among them as our concrete security level (including classical and quantum hardness).

Table 3: Classical and quantum hardness of the different parameter sets of FLIT.

		FLIT128	FLIT256	FLIT512
RLWE	Classical	134.0	269.5	529.1
	Quantum	129.7	253.2	488.5
NTRU	Classical	139.4	281.2	554.3
	Quantum	133.7	261.1	506.9

The estimates in Table 3 are obtained using the MATZOV [16] model for BKZ cost estimation, which is the default cost model in the version of the Lattice Estimator employed. We take the minimum value across all attack strategies (RLWE and NTRU, classical and quantum) as the final stated security level for each parameter set; these minimal values are shown in bold.

To evaluate the security of FLIT more comprehensively, we further report estimates obtained specifically for the primal uSVP attack, which is the most relevant attack in the lattice setting. We consider two cost models: the Core-SVP methodology provides a rough operation-count estimate (Table 4), while the MATZOV model yields a more detailed gate-count estimate (Table 5).

Table 4: Core-SVP methodology, Primal attack only based on lattice attack dimension (d) and BKZ blocksize (β), reported separately for RLWE and NTRU instances.

		FLIT128	FLIT256	FLIT512
RLWE	lattice attack dim. d	861	1727	3570
	BKZ-blocksize β	407	915	1885
	Core-SVP classical hardness	118.8	267.2	550.4
	Core-SVP quantum hardness	104.6	235.2	484.4
NTRU	lattice attack dim. d	884	1710	3622
	BKZ-blocksize β	406	915	1884
	Core-SVP classical hardness	118.6	267.2	550.1
	Core-SVP quantum hardness	104.3	235.2	484.2

Table 5: Lattice estimator (MATZOV model) estimated gates for primal attack

		FLIT128	FLIT256	FLIT512
RLWE	$\log_2(\text{gates})$	143.7	286.3	560
NTRU	$\log_2(\text{gates})$	143.5	286.3	559.8

4 Performance Evaluation

4.1 Performance Analysis

Thanks to efficiency of NTT-based polynomial multiplication, the dominant factor in FLIT’s performance is the symmetric primitive—particularly the hash functions H and G , the key derivation function KDF, and the pseudorandom sampling of ternary polynomials. All of these are built exclusively on the auxiliary hash function provided with the submission package.

Symmetric invocation profile. Table 6 summarizes the number of symmetric primitive calls in each FLIT.CCAKEM operation. The ternary sampling functions f, g, r, e each internally invoke the PRF to expand a seed into a full polynomial, and are therefore counted as symmetric operations.

Table 6: Number of symmetric primitive calls per FLIT.CCAKEM operation.

Operation	H	G	KDF	f	g	r	e
FLIT.CCAKEM.KeyGen	1	0	0	1	1	0	0
FLIT.CCAKEM.Enc	3	1	1	0	0	1	1
FLIT.CCAKEM.Dec	1	1	1	0	0	1	1

Impact on key generation. FLIT.CCAKEM.KeyGen invokes H once (to hash the public key into the secret key for the FO transform), and samples two ternary polynomials f and g via the PRF. In the reference implementation, these symmetric primitives account for approximately 16–21% of the total KeyGen cycles across all three security levels,³ while the other major contributor is the inversion of the secret polynomial f (in both $\mathbb{F}_2[x]/(x^n + 1)$ and the NTT domain). In the AVX2-optimized implementation, where NTT and polynomial arithmetic have been accelerated by up to 50×, the symmetric proportion rises to 35–49%, making the auxiliary hash function the primary bottleneck in key generation.

Impact on encapsulation. FLIT.CCAKEM.Enc invokes H three times (self-hashing of the random seed, hashing of the public key, and hashing of the ciphertext), G once (to derive the shared key and encryption coins), KDF once (to produce the final shared secret), and samples two ternary polynomials r and e . In the reference implementation, these symmetric calls contribute 44–54% of the total encapsulation cycles. In the AVX2 implementation, the NTT acceleration shrinks the non-symmetric portion to near-negligible levels, pushing the symmetric share to 88–97%—the encapsulation operation is effectively bounded by the throughput of the auxiliary hash function.

Impact on decapsulation. FLIT.CCAKEM.Dec invokes H once (hashing the ciphertext), G once (re-deriving the pre-key and coins for re-encryption), KDF once, and samples r and e for the FO re-encryption check. The symmetric primitives account for 19–26% of the reference decapsulation cycles and 59–73% of the AVX2 decapsulation cycles.

Across the three security levels, the performance of the symmetric primitives constitute 16–54% of total KEM cycles in the reference implementation, and 35–97% in the AVX2 implementation. This gap reflects the fact that AVX2 acceleration disproportionately benefits the non-symmetric components (NTT, polynomial arithmetic, decoding) while leaving the symmetric primitives as the remaining performance bottleneck—any further speedup of FLIT would require accelerating the hash function itself.

³All percentage figures reported in this section are derived from the average cycle counts measured on the evaluation platform described in Section 4.2. The exact proportions depend on the relative speeds of the auxiliary hash function and the NTT arithmetic on a given microarchitecture and may vary across platforms.

4.2 Performance Test

To evaluate the performance of FLIT, we benchmark its reference implementation written in pure C and compiled using GCC 11.4.0 with `-O3 -march=native -fomit-frame-pointer`. The AVX2-optimized implementation is compiled using the same compiler with `-O2 -mavx2 -mbmi2 -mpopcnt -mpclmul -march=native -mtune=native -fomit-frame-pointer`.

All benchmarks are conducted on an x86_64 platform equipped with an AMD Ryzen 7 5700X3D CPU (8 cores / 16 threads, Zen 3 microarchitecture, base frequency 3.0 GHz, boost up to 4.1 GHz). The system features 16 GB DDR4 RAM and runs Ubuntu 22.04.5 LTS on WSL2. The AVX2-optimized implementation leverages the AVX2, BMI2, POPCNT, and PCLMUL instruction sets available on this processor. No external cryptographic library is used apart from the auxiliary hash function provided with the submission package.

4.2.1 Reference Implementation

Table 7 reports the performance results of the reference implementation utilizing SM3. The reported values correspond to average cycle counts over 10,000 executions of the FLIT scheme.

Table 7: **Reference implementation speeds:** average cycle counts over 10 000 executions for FLIT

Scheme	KeyGen	Enc	Dec
FLIT128	124 502	108 650	148 369
FLIT256	315 646	231 208	343 472
FLIT512	1 311 984	843 577	1 182 405

4.2.2 AVX2 Optimized Implementation

The efficiency of the scheme is primarily determined by the cost of Number Theoretic Transform (NTT) operations and related polynomial arithmetic. To improve performance, we introduce an AVX2-based optimization that exploits SIMD parallelism to accelerate polynomial multiplication and modular reduction. Table 8 presents the performance of the AVX2-optimized implementation under SM3. All results are reported as average cycle counts over 10,000 executions of the FLIT scheme.

Table 8: **AVX2 implementation speeds:** average cycle counts over 10 000 executions for FLIT

Scheme	KeyGen	Enc	Dec
FLIT128	42 698	45 706	41 867
FLIT256	101 037	83 035	78 797
FLIT512	479 191	339 547	320 925

4.2.3 Additional Implementation

Since the symmetric primitives dominate the running time of FLIT, we provide an additional implementation for FLIT128 and FLIT256 using functions standardized in FIPS-202 [7], in order to give a more direct indication of the scheme’s speed. We do not provide such an implementation for FLIT512, as it requires a hash function with 512-bit security, which is not available in FIPS-202. Concretely, the primitives are instantiated as follows:

- H is instantiated with SHA-256;
- G is instantiated with SHA-512;
- PRF and KDF are instantiated with SHAKE-256.

For the FIPS-202 instantiation, we provide both a reference implementation (see Table 9) and an optimized implementation (see Table 10), measured under the same benchmarking setup as above. Owing to the use of the faster symmetric primitives, these implementations achieve higher speeds than

the corresponding reference and optimized implementations described in 4.2.1 and 4.2.2. These results are reported for reference only, and are solely intended to show that the performance of FLIT can be substantially improved with faster symmetric primitives.

Table 9: **Additional implementation speeds (REF):** average cycle counts over 10 000 executions for FLIT.

Scheme	KeyGen	Enc	Dec
FLIT128	94 230	67 005	118 452
FLIT256	255 172	166 032	298 492

Table 10: **Additional implementation speeds (AVX2):** average cycle counts over 10 000 executions for FLIT.

Scheme	KeyGen	Enc	Dec
FLIT128	29 568	26 629	24 073
FLIT256	65 612	48 033	49 216

4.2.4 Communication and Storage Overhead

Table 11 summarizes the sizes of secret keys, public keys, and ciphertexts for the three FLIT parameter sets.

The secret key stores the private polynomial f in uncompressed NTT-domain form, so that it can be used directly in the NTT-domain multiplication during decryption without an extra forward transform. It also contains the $\mathbb{Z}_2[X]/(X^n + 1)$ inverse of f (for decoding), a copy of the public key, and an FO-transform seed. The public key $h = g \cdot f_{\text{NTT}}^{-1}$ is stored in NTT-domain representation. For FLIT128 and FLIT256, where $q = 769$, the NTT-domain coefficients admit a compact encoding that exploits the gap between q and the next power of two, reducing the public key below the naive $\lceil \log_2 q \rceil \cdot N$ bits. For FLIT512 ($q = 3329$), no such redundancy is available and the coefficients are stored at full width. The ciphertext is the data that will be transmitted, it is compressed to reduce bandwidth. Compression truncates the low-order bits of each coefficient, which is only meaningful in the standard coefficient domain; in the NTT domain the coefficients are roughly random modulo q and no low-order truncation is possible. Ciphertexts are therefore serialized directly from the standard domain.

Table 11: Secret key, public key and ciphertext sizes in bytes for FLIT.

Scheme	Secret Key	Public Key	Ciphertext
FLIT128	839	615	512
FLIT256	1581	1229	1024
FLIT512	3776	3072	2304

5 Key Properties

The design of FLIT is guided by three principles: structural simplicity, robustness against decryption failures, and efficient decodability. Together they yield a scheme that is compact, easy to implement, and well-suited for constrained environments.

Simplicity. FLIT follows a simple NTRU structure, forming the ciphertext as $hr + e + \frac{q+1}{2}\text{Encode}(m)$. The plaintext encoding uses a repetition code, which requires no polynomial ring arithmetic and thus keeps encryption both efficient and implementation friendly.

Lower decryption failure probability. We leverage the redundancy introduced by repetition codes to substantially lower the decryption failure probability. This improved robustness provides greater flexibility in parameter selection, allowing more aggressive choices (e.g., larger noise or a smaller modulus) while still preserving correctness. As a result, the overall key and ciphertext sizes can be reduced, leading to a more compact and efficient scheme.

Homomorphic encoding and decoding. We introduce a homomorphically compatible encoding that preserves multiplication under the secret key modulo 2. Combined with a repetition structure, this enables direct recovery of $f \cdot m$ via simple thresholding, followed by a lightweight inversion step. This design yields a simple, efficient, and highly robust decoding procedure, and it represents a key innovation of our construction.

Compatibility and extensibility. FLIT operates over standard power-of-two cyclotomic rings and uses conventional NTT-based polynomial arithmetic, making it compatible with existing lattice-based cryptography infrastructure and readily portable across platforms. The symmetric layer is fully self-contained, requiring no external cryptographic library. The parameter framework is naturally extensible: the ring dimension N , the modulus q , the repetition factor, and the ternary noise distributions can all be scaled to meet higher security targets without altering the core NTRU structure or the encoding and decoding procedures.

References

- [1] Albrecht, M.R., Cid, C., Faugere, J.C., Fitzpatrick, R., Perret, L.: On the complexity of the bkw algorithm on lwe. *Designs, Codes and Cryptography* 74(2), 325–354 (2015)
- [2] Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* 9(3), 169–203 (2015)
- [3] Alkim, E., Avanzi, R., Bos, J., Ducas, L., de la Piedra, A., Pöppelmann, T., Schwabe, P., Stebila, D., Albrecht, M.R., Orsini, E., et al.: Newhope—submission to the nist post-quantum project. Specification document (part of the submission package) (2019)
- [4] Arora, S., Ge, R.: New algorithms for learning in presence of errors. In: *International Colloquium on Automata, Languages, and Programming*. pp. 403–415. Springer (2011)
- [5] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D., et al.: Crystals-kyber algorithm specifications and supporting documentation. *NIST PQC Round 2(4)*, 1–43 (2019)
- [6] Chen, Y., Nguyen, P.Q.: Bkz 2.0: Better lattice security estimates. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 1–20. Springer (2011)
- [7] Dworkin, M.J., et al.: Sha-3 standard: Permutation-based hash and extendable-output functions (2015)
- [8] D’Anvers, J.P., Rossi, M., Virdia, F.: (one) failure is not an option: bootstrapping the search for failures in lattice-based encryption schemes. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 3–33. Springer (2020)
- [9] Fouque, P.A., Kirchner, P., Pornin, T., Yu, Y.: Bat: Small and fast kem over ntru lattices. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 240–265 (2022)
- [10] Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: *Annual international cryptology conference*. pp. 537–554. Springer (1999)
- [11] Guo, Q., Johansson, T.: Faster dual lattice attacks for solving lwe with applications to crystals. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 33–62. Springer (2021)
- [12] Hoffstein, J., Pipher, J., Silverman, J.H.: Ntru: A ring-based public key cryptosystem. In: *International algorithmic number theory symposium*. pp. 267–288. Springer (1998)
- [13] Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the fujisaki-okamoto transformation. In: *Theory of Cryptography Conference*. pp. 341–371. Springer (2017)
- [14] Howgrave-Graham, N.: A hybrid lattice-reduction and meet-in-the-middle attack against ntru. In: *Annual International Cryptology Conference*. pp. 150–169. Springer (2007)
- [15] Jiang, H., Zhang, Z., Chen, L., Wang, H., Ma, Z.: Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In: *Annual International Cryptology Conference*. pp. 96–125. Springer (2018)
- [16] MATZOV: Report on the security of lwe: Improved dual lattice attack (2022), <https://doi.org/10.5281/zenodo.6412487>
- [17] Pöppelmann, T., Güneysu, T.: Towards practical lattice-based public-key encryption on reconfigurable hardware. In: *International Conference on Selected Areas in Cryptography*. pp. 68–85. Springer (2013)
- [18] Saito, T., Xagawa, K., Yamakawa, T.: Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 520–551. Springer (2018)

- [19] Stehlé, D., Steinfeld, R.: Making ntru as secure as worst-case problems over ideal lattices. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 27–47. Springer (2011)
- [20] Zhang, J., Feng, D., Yan, D.: Nev: faster and smaller ntru encryption using vector decoding. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 157–189. Springer (2023)